```python
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

```
Using device: cpu
```

```python
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
```

```
100%|██████████| 170M/170M [00:03<00:00, 48.9MB/s]
```

```python
test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transform
)

batch_size = 128

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False
)

class SparseAutoencoder(nn.Module):

    def __init__(self, input_dim=3072, hidden_dim=512):
        super(SparseAutoencoder, self).__init__()
```

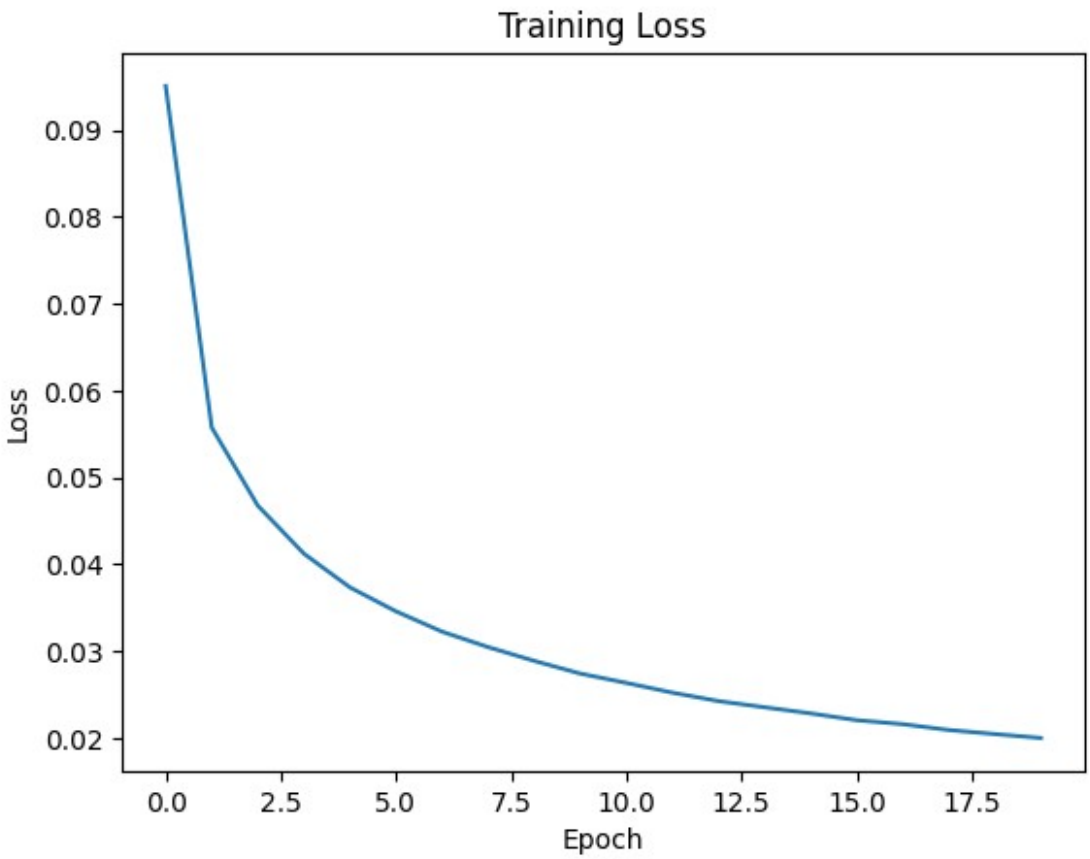
```
        # Encoder
        self.encoder = nn.Sequential(
            nn.Linear(input_dim,1024),
            nn.ReLU(),
            nn.Linear(1024,hidden_dim),
            nn.Sigmoid()
        )

        # Decoder
        self.decoder = nn.Sequential(
            nn.Linear(hidden_dim,1024),
            nn.ReLU(),
            nn.Linear(1024,input_dim),
            nn.Tanh()
        )

    def forward(self,x):

        encoded = self.encoder(x)

        decoded = self.decoder(encoded)

        return encoded, decoded

def kl_divergence(rho, rho_hat):

    eps = 1e-8

    rho_hat = torch.mean(rho_hat, dim=0)

    rho_hat = torch.clamp(rho_hat, eps, 1-eps)

    rho = torch.tensor([rho]*len(rho_hat)).to(device)

    kl = rho * torch.log(rho/rho_hat) + \
         (1-rho) * torch.log((1-rho)/(1-rho_hat))

    return torch.sum(kl)

model = SparseAutoencoder().to(device)

criterion = nn.MSELoss()

optimizer = optim.Adam(model.parameters(), lr=0.0005)

epochs = 20

rho = 0.05
beta = 1e-4
```

```python
loss_history = []

for epoch in range(epochs):

    total_loss = 0

    for images,_ in train_loader:

        images = images.to(device)

        images = images.view(images.size(0),-1)

        encoded, decoded = model(images)

        recon_loss = criterion(decoded,images)

        sparsity_loss = kl_divergence(rho,encoded)

        loss = recon_loss + beta * sparsity_loss

        optimizer.zero_grad()

        loss.backward()+

        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss/len(train_loader)

    loss_history.append(avg_loss)

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}")
```

```
Epoch [1/20], Loss: 0.0951
Epoch [2/20], Loss: 0.0557
Epoch [3/20], Loss: 0.0467
Epoch [4/20], Loss: 0.0412
Epoch [5/20], Loss: 0.0373
Epoch [6/20], Loss: 0.0346
Epoch [7/20], Loss: 0.0322
Epoch [8/20], Loss: 0.0304
Epoch [9/20], Loss: 0.0289
Epoch [10/20], Loss: 0.0274
Epoch [11/20], Loss: 0.0263
Epoch [12/20], Loss: 0.0252
Epoch [13/20], Loss: 0.0242
Epoch [14/20], Loss: 0.0235
Epoch [15/20], Loss: 0.0228
Epoch [16/20], Loss: 0.0220
Epoch [17/20], Loss: 0.0216
```

```
Epoch [18/20], Loss: 0.0209
Epoch [19/20], Loss: 0.0204
Epoch [20/20], Loss: 0.0200
```

```
plt.plot(loss_history)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```




```
model.eval()

test_loss = 0

with torch.no_grad():

    for images, _ in test_loader:

        images = images.to(device)

        images = images.view(images.size(0), -1)

        encoded, decoded = model(images)
```

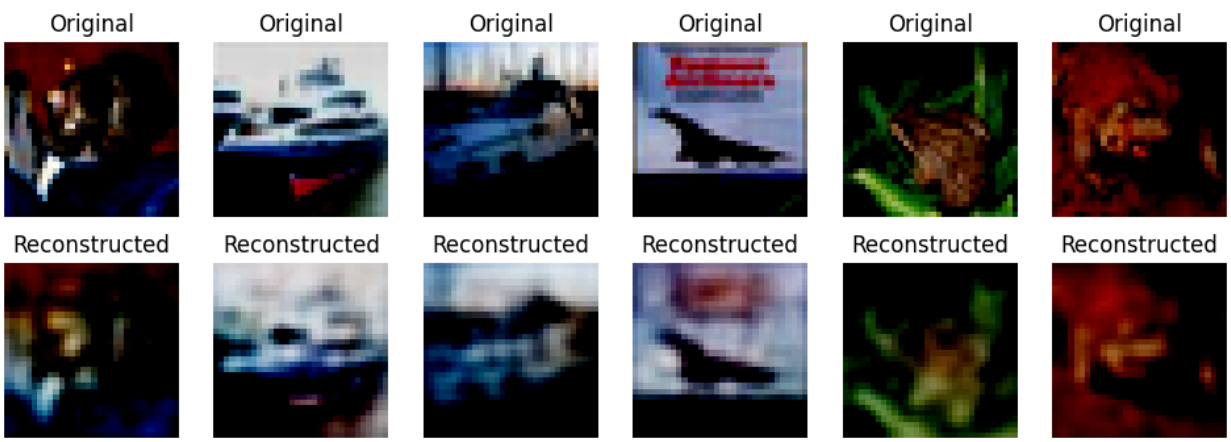
```
        loss = criterion(decoded, images)

        test_loss += loss.item()

print("Test Reconstruction Loss:", test_loss / len(test_loader))
```

```
Test Reconstruction Loss: 0.02061362446674818
```

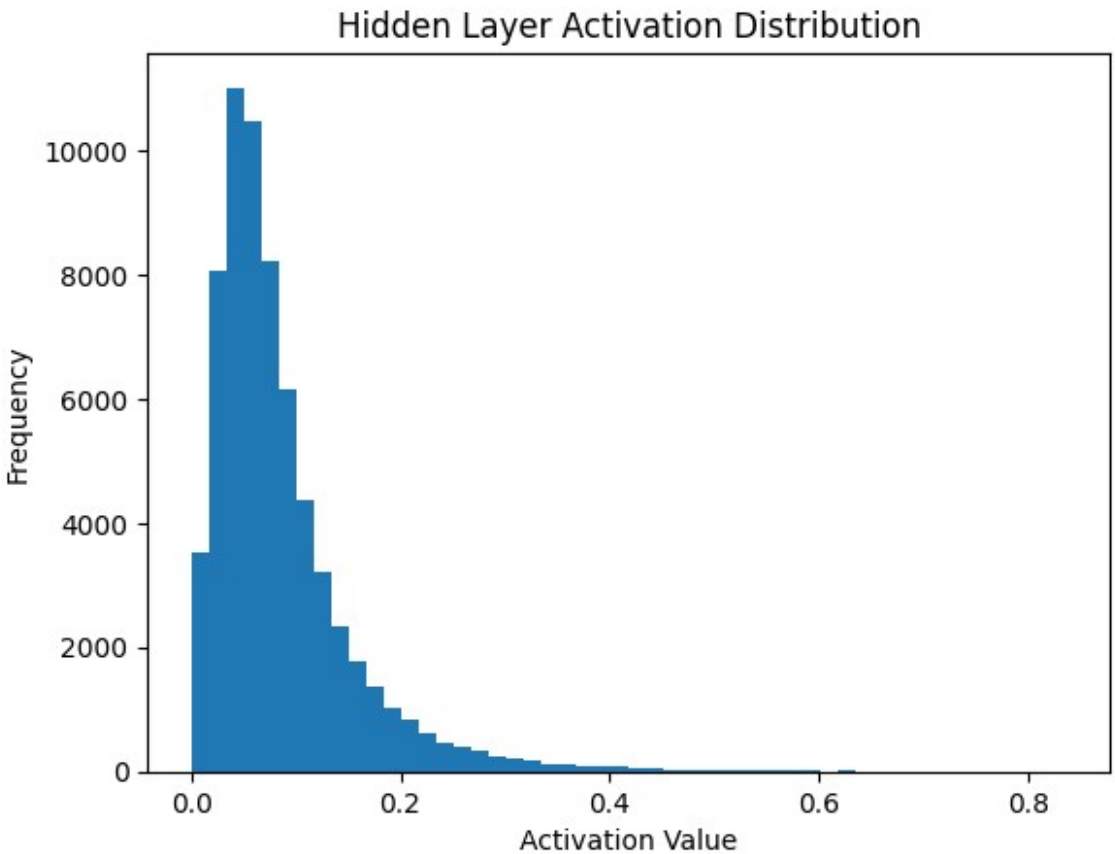
```
dataiter = iter(test_loader)

images, _ = next(dataiter)

images = images.to(device)

flat = images.view(images.size(0), -1)

encoded, outputs = model(flat)

outputs = outputs.view(-1,3,32,32).cpu()
images = images.cpu()

fig, axes = plt.subplots(2,6, figsize=(12,4))

for i in range(6):

    axes[0,i].imshow(np.transpose(images[i], (1,2,0)))
    axes[0,i].set_title("Original")
    axes[0,i].axis("off")

    axes[1,i].imshow(np.transpose(outputs[i].detach(), (1,2,0)))
    axes[1,i].set_title("Reconstructed")
    axes[1,i].axis("off")

plt.show()
```

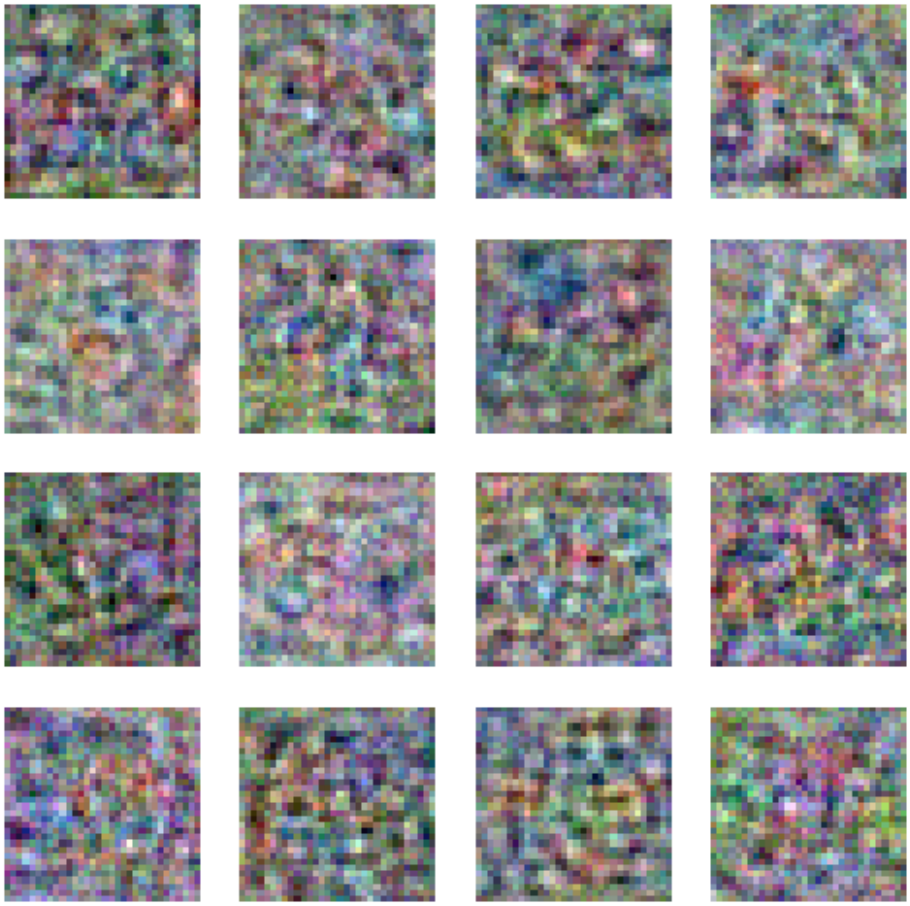
```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.8980392..1.0].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.81292..0.80765015].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-1.0..0.9372549].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.9145411..0.89767367].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.9764706..0.9764706].
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.82677215..0.92845184].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.7411765..0.8352941].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.67437375..0.8260617].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.9529412..0.7254902].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.78516424..0.5613578].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-1.0..0.654902].
WARNING:matplotlib.image:Clipping input data to the valid range for
imshow with RGB data ([0..1] for floats or [0..255] for integers). Got
range [-0.86576444..0.6141095].
```




```
encoded = encoded.cpu().detach().numpy()

plt.hist(encoded.flatten(), bins=50)
plt.title("Hidden Layer Activation Distribution")
plt.xlabel("Activation Value")
plt.ylabel("Frequency")
plt.show()
```




```
weights = model.encoder[0].weight.data.cpu()

fig, axes = plt.subplots(4,4, figsize=(6,6))

for i, ax in enumerate(axes.flat):

    w = weights[i].reshape(3,32,32)

    w = (w - w.min())/(w.max()-w.min())

    ax.imshow(np.transpose(w,(1,2,0)))

    ax.axis('off')

plt.show()
```